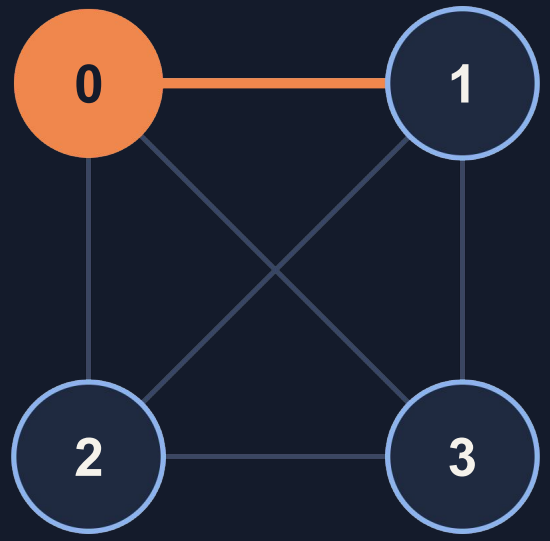
LECTURE 7 · CS-401 PARALLEL AND DISTRIBUTED COMPUTING

Message Passing Interface (MPI)

How parallel processes communicate — with C examples you can run in Google Colab.



INTRODUCTION

What is MPI?

MPI — the Message Passing Interface — is a standard for communication between parallel processes.

High-performance computing

Widely used in HPC to run large parallel programs.

One computer or many

Runs parallel programs on a single machine or across multiple machines.

C, C++ and Fortran

The languages MPI is most commonly used with.

MOTIVATION

Why do we need MPI?

01

Divide

Large problems are slow sequentially, so split them into smaller tasks.

02

Work in parallel

Processes work on different tasks at the same time.

03

Exchange

Processes share data with each other when required.

04

Combine

Partial results are merged, reducing total time.

Example: add one million numbers using four processes

RANK 0	RANK 1	RANK 2	RANK 3
1 – 250,000	250,001 – 500,000	500,001 – 750,000	750,001 – 1,000,000

Each rank computes a **partial sum** → the four partial sums are combined into the **final answer**.

CORE IDEA

What is message passing?

Process 0
Sends the value 100



Process 1
Receives the value 100

Send
Transmit data to another process.

Receive
Accept data from another process.

Source
The process sending the message.

Destination
The process receiving the message.

MEMORY MODELS

Shared memory vs distributed memory

Shared memory

MEMORY

Processes or threads can access common memory.

COMMUNICATION

Can use shared variables.

COMMON MODEL

OpenMP

Distributed memory

MEMORY

Each process has its own private memory space.

COMMUNICATION

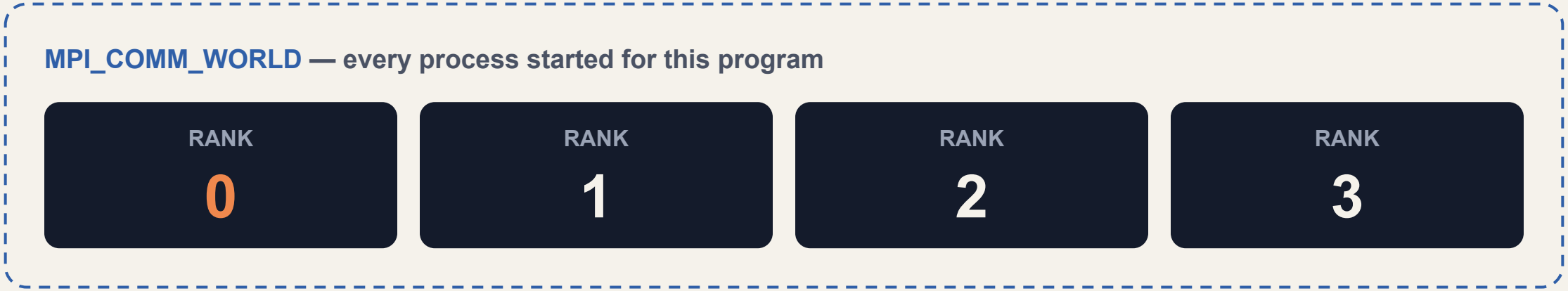
Commonly uses messages.

COMMON MODEL

MPI

ARCHITECTURE

Processes, ranks and MPI_COMM_WORLD



Each process gets a **unique rank** within a communicator, usually starting from 0.

The program uses ranks to **decide which process** performs which task.

A rank is an ID inside MPI — **not an IP address**.

THE MPI API

Six functions to get started

STAGE	FUNCTION	PURPOSE
Setup	**MPI_Init()**	Initializes the MPI environment.
Identify	**MPI_Comm_rank()**	Gets the rank of the calling process.
Identify	**MPI_Comm_size()**	Gets the number of processes in a communicator.
Communicate	**MPI_Send()**	Sends a message.
Communicate	**MPI_Recv()**	Receives a message.
Shut down	**MPI_Finalize()**	Ends the MPI environment.

SENDING A MESSAGE

Understanding MPI_Send()

```
MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
```

&value	buffer	Address of the data to send.
1	count	Number of data items.
MPI_INT	datatype	The data type is integer.
1	destination	**Destination rank** — who receives the message.
0	tag	Message tag — a label for the message.
MPI_COMM_WORLD	communicator	The group of processes involved.

RECEIVING A MESSAGE

Understanding MPI_Recv()

```
MPI_Recv(&value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

&value	buffer	Memory location where the data is stored.
1	count	Maximum number of integers expected.
MPI_INT	datatype	Expected data type.
0	source	**Source rank** — who the message comes from.
0	tag	Message tag — must match the sender's tag.
MPI_COMM_WORLD	communicator	The group of processes involved.
MPI_STATUS_IGNORE	status	Ignore extra message-status details.

PUTTING IT TOGETHER

A complete MPI program in C

Rank 0 sets value = 100 and sends it to rank 1.

Rank 1 receives the value and prints it.

Both reach MPI_Finalize() before exiting.

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rank, value;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 0) {
        value = 100;
        printf("Process 0: Sending %d\n", value);
        MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    } else if (rank == 1) {
        MPI_Recv(&value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
                 MPI_STATUS_IGNORE);
        printf("Process 1: Received %d\n", value);
    }

    MPI_Finalize();
    return 0;
}
```

HANDS-ON

Running MPI in Google Colab

01 **Install MPI**

```
!apt-get update -qq
!apt-get install -y openmpi-bin libopenmpi-dev
```

02 **Save the program**

```
%%writefile mpi_demo.c ← first line of the cell, then the C code
```

03 **Compile**

```
!mpicc mpi_demo.c -o mpi_demo
```

04 **Run 2 processes**

```
!mpirun --allow-run-as-root --oversubscribe -np 2 ./mpi_demo
```

EXPECTED OUTPUT

Two processes, one message

Process 0: Sending 100

Process 1: Received 100

mpicc
Compiles the MPI program.

mpirun
Launches the processes.

-np 2
Requests two processes.

--oversubscribe
Allows more processes than available slots.

REFLECTION

Why is this program important?

Multiple processes

The same program runs as two separate processes.

Communication

One process sends data, and another receives it.

Process identification

Rank 0 and rank 1 decide which process sends and which receives.

Needs **at least 2 processes** because it sends to rank 1: run with -np 2.

Ask students: what happens if Process 0 sends 500 instead of 100?

COMMUNICATION PATTERNS

Point-to-point communication

- Happens between **exactly two processes**.
- One process **sends**; another process **receives**.
- Uses **MPI_Send()** and **MPI_Recv()**.
- Source, destination and message details must be **compatible**.



COMMUNICATION PATTERNS

Collective communication

Involves every process in a communicator — not just two.

MPI_Bcast()

Distributes the same data from one process to all processes.

MPI_Scatter()

Splits data into portions and gives one portion to each process.

MPI_Gather()

Collects data from all processes at one process.

MPI_Reduce()

Combines values from all processes using an operation, such as a sum.

Send data out with Bcast and Scatter · **bring results back** with Gather and Reduce.

DEMONSTRATION 2

Divide the problem: a parallel sum

INPUT ON PROCESS 0

10	20	30	40	50	60	70	80

↓ MPI_Scatter() — two numbers to each process

PROCESS 0
10 + 20
= 30

PROCESS 1
30 + 40
= 70

PROCESS 2
50 + 60
= 110

PROCESS 3
70 + 80
= 150

MPI_Reduce() with MPI_SUM → **30 + 70 + 110 + 150 = 360**

DEMONSTRATION 2 · MPI_SUM.C

The complete parallel-sum program

```c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rank, size;
    int numbers[8] = {10, 20, 30, 40,
                      50, 60, 70, 80};
    int local[2];
    int local_sum = 0, total_sum = 0;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 4) {
        if (rank == 0)
            printf("Please run with -np 4\n");
        MPI_Finalize();
        return 0;
    }
```

```c
    /* 1. Give two numbers to each process */
    MPI_Scatter(numbers, 2, MPI_INT,
                local, 2, MPI_INT,
                0, MPI_COMM_WORLD);

    /* 2. Each process adds its own pair */
    local_sum = local[0] + local[1];
    printf("Process %d: %d + %d = %d\n",
           rank, local[0], local[1], local_sum);

    /* 3. Combine partial sums at rank 0 */
    MPI_Reduce(&local_sum, &total_sum, 1, MPI_INT,
               MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0)
        printf("Final sum = %d\n", total_sum);

    MPI_Finalize();
    return 0;
}
```

DEMONSTRATION 2

Run it with four processes

01 **Save**

```
%%writefile mpi_sum.c ← first line of the code cell
```

02 **Compile**

```
!mpicc mpi_sum.c -o mpi_sum
```

03 **Run 4 processes**

```
!mpirun --allow-run-as-root --oversubscribe -np 4 ./mpi_sum
```

```
Process 0: 10 + 20 = 30
Process 1: 30 + 40 = 70
Process 2: 50 + 60 = 110
Process 3: 70 + 80 = 150
Final sum = 360
```

The order of the **Process** lines can change from run to run. The final sum is always **360**.

DEMONSTRATION 2

How the processes cooperate

STEP 1 · DISTRIBUTE

MPI_Scatter()

Process 0 splits the eight numbers; each process receives two.

STEP 2 · COMPUTE

Local calculation

Each process adds its two numbers independently, e.g. Process 2: 50 + 60 = 110.

STEP 3 · COMBINE

MPI_Reduce()

MPI adds the four partial sums with MPI_SUM and stores 360 at Process 0.

MPI_Scatter() distributes the input; MPI_Reduce() combines the results. Together they let processes cooperate on one problem.

TROUBLESHOOTING

“mpirun was unable to launch ./mpi_sum”

The executable does not exist: the **%%writefile** cell was not run, or **compilation failed**. Check each step in order:

01	**Source file exists?**	`!ls -l mpi_sum.c`
02	**Compile, read errors**	`!mpicc mpi_sum.c -o mpi_sum && echo "Compilation successful"`
03	**Executable exists?**	`!ls -l mpi_sum`
04	**Run again**	`!mpirun --allow-run-as-root --oversubscribe -np 4 ./mpi_sum`

Also check: **!which mpicc** — don't mix MPICH and Open MPI; compile and run with the same one.

RECAP

Key takeaways

1. MPI is a **standard** for communication between parallel processes, on one computer or many.
2. Each process has a **rank**; **MPI_COMM_WORLD** groups all processes of the program.
3. **MPI_Send()** and **MPI_Recv()** move data between two specific processes.
4. **Scatter** splits the work and **Reduce** combines the results: 8 numbers, 4 processes, sum 360.

Questions?